Simple Dictionary

Part 1 — Data, state and storage

Phone edition • Source snapshot: 28 October 2026

How to use this document

Each source line has its own label, code and explanation. This edition includes imports, blank lines and closing brackets too.

Code is above its explanation rather than beside it. Leading indentation is removed and long code is wrapped to fit a phone. A wrapped code block is still one original source line.

Line numbers refer to the HTML walkthrough snapshot. Later source edits can change them. The existing API key is hidden in the guide only; app source is unchanged.

Start with BeginnerGuide.pdf for the overall search, favorite and preference flows. Use this document when you want to understand a particular file or line.

The file list below and PDF bookmarks help navigation. Some platform/Room/browser setup goes beyond the modules' short examples. Generated files, binary assets and Xcode project internals are outside the handwritten-source scope.

This volume: 15 files; 495 annotated source lines.

Files in reading order

1. shared/src/commonMain/kotlin/com/example/dictionary/domain/Models.kt
2. shared/src/commonMain/kotlin/com/example/dictionary/presentation/UiState.kt
3. shared/src/commonMain/kotlin/com/example/dictionary/domain/DictionaryRepository.kt
4. shared/src/commonMain/kotlin/com/example/dictionary/domain/SearchWordUseCase.kt
5. shared/src/commonMain/kotlin/com/example/dictionary/presentation/DictionaryViewModel.kt
6. shared/src/commonMain/kotlin/com/example/dictionary/data/DictionaryRepository.kt
7. shared/src/commonMain/kotlin/com/example/dictionary/domain/HistoryUseCase.kt

8. shared/src/commonMain/kotlin/com/example/dictionary/domain/PreferencesRepository.kt
9. shared/src/commonMain/kotlin/com/example/dictionary/domain/SlangPreferenceUseCase.kt
10. shared/src/commonMain/kotlin/com/example/dictionary/data/local/HistoryEntity.kt
11. shared/src/commonMain/kotlin/com/example/dictionary/data/local/HistoryDao.kt
12. shared/src/commonMain/kotlin/com/example/dictionary/data/local/DictionaryDatabase.kt
13. shared/src/commonMain/kotlin/com/example/dictionary/data/DictionaryPreferences.kt
14. shared/src/commonMain/kotlin/com/example/dictionary/data/AppStorage.kt
15. shared/src/commonMain/kotlin/com/example/dictionary/data/HttpClientFactory.kt

1. Models.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/Models.kt

The app’s containers for definitions, saved-history display items and complete lookup answers.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.serialization.Serializable
```

Makes Serializable available here from kotlinx.serialization.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 6

```
data class DictionaryEntry(
```

Defines DictionaryEntry as a data container.

Line 7

```
val word: String,
```

The word associated with this object.

Line 8

```
val source: String,
```

The dictionary that supplied the definition.

Line 9

```
val partOfSpeech: String = "",
```

A grammar label such as noun; empty when unavailable.

Line 10

```
val definition: String,
```

The meaning to display.

Line 11

```
val example: String = ""
```

An optional example sentence; empty when unavailable.

Line 12

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 13

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 14

```
data class HistoryItem(
```

Defines HistoryItem as a data container.

Line 15

```
val word: String,
```

The word associated with this object.

Line 16

```
val snippet: String,
```

A short preview of a saved definition for History.

Line 17

```
val formalEntries: List<DictionaryEntry>,
```

The list of formal dictionary definitions.

Line 18

```
val urbanEntries: List<DictionaryEntry>,
```

The list of slang definitions.

Line 19

```
val isFavorite: Boolean = false
```

Whether this history word is starred; false is the default.

Line 20

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
// A named result is easier to understand than
Triple.first/second/third.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 23

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 24

```
data class DictionaryResult(
```

Defines DictionaryResult as a data container.

Line 25

```
val formalEntries: List<DictionaryEntry>,
```

The list of formal dictionary definitions.

Line 26

```
val urbanEntries: List<DictionaryEntry>,
```

The list of slang definitions.

Line 27

```
val didYouMeanSuggestion: String? = null,
```

An optional corrected spelling; null means there is no suggestion.

Line 28

```
val isCached: Boolean = false
```

Whether this returned result came from the saved cache; false is the default.

Line 29

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

2. UiState.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/presentation/UiState.kt

All possible result-screen conditions in one sealed family.

Line 1

```
package com.example.dictionary.presentation
```

Places this file in the com.example.dictionary.presentation namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.domain.DictionaryEntry
```

Makes DictionaryEntry available here from com.example.dictionary.domain.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
sealed class UiState {
```

Limits result-screen states to the classes declared in this family.

Line 6

```
object Idle : UiState()
```

The user has not selected an active result yet.

Line 7

```
object Loading : UiState()
```

The app is currently waiting for a lookup.

Line 8

```
data class Success(
```

A successful state carries formal entries, slang entries and an optional suggestion.

Line 9

```
val formalEntries: List<DictionaryEntry>,
```

The list of formal dictionary definitions.

Line 10

```
val urbanEntries: List<DictionaryEntry>,
```

The list of slang definitions.

Line 11

```
val didYouMeanSuggestion: String? = null
```

An optional corrected spelling; null means there is no suggestion.

Line 12

```
) : UiState()
```

Finishes the Success data-class constructor and makes Success one of the permitted UiState cases.

Line 13

```
data class Error(val message: String) : UiState()
```

An error state carries the message that the screen should display.

Line 14

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

3. DictionaryRepository.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/DictionaryRepository.kt

The contract for lookups, observable history and favorite changes. It does not make HTTP calls itself.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
// Module 6: presentation depends on an interface, not
Ktor or Room.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 6

```
interface DictionaryRepository {
```

Defines DictionaryRepository as an interface of promised operations.

Line 7

```
suspend fun fetchDefinitions(word: String, forceRefresh:
Boolean = false): DictionaryResult?
```

Defines fetchDefinitions(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 8

```
fun observeHistory(): Flow<List<HistoryItem>>
```

Defines observeHistory(). Inputs are inside parentheses; the return type describes its answer.

Line 9

```
suspend fun toggleFavorite(word: String)
```

Defines toggleFavorite(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

4. SearchWordUseCase.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/SearchWordUseCase.kt

One small rule: normalize the query, then request its definitions.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
class SearchWordUseCase(private val repository:
DictionaryRepository) {
```

Receives the repository interface instead of constructing a data source itself.

Line 4

```
suspend operator fun invoke(word: String, forceRefresh:
Boolean = false): DictionaryResult? {
```

Allows an instance to be called like searchWordUseCase(word). The function can pause and accepts an optional forceRefresh flag.

Line 5

```
return
repository.fetchDefinitions(word.trim().lowercase(),
forceRefresh)
```

Cleans the input, asks the repository to fetch it, and sends its answer back unchanged.

Line 6

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 7

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

5. DictionaryViewModel.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/presentation/DictionaryViewModel.kt

Controls screen state and asynchronous actions. It observes persisted data and publishes state/events.

Line 1

```
package com.example.dictionary.presentation
```

Places this file in the com.example.dictionary.presentation namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.domain.DictionaryResult
```

Makes DictionaryResult available here from com.example.dictionary.domain.

Line 4

```
import com.example.dictionary.domain.HistoryItem
```

Makes HistoryItem available here from com.example.dictionary.domain.

Line 5

```
import com.example.dictionary.domain.HistoryUseCase
```

Makes HistoryUseCase available here from com.example.dictionary.domain.

Line 6

```
import com.example.dictionary.domain.SearchWordUseCase
```

Makes SearchWordUseCase available here from com.example.dictionary.domain.

Line 7

```
import
com.example.dictionary.domain.SlangPreferenceUseCase
```

Makes SlangPreferenceUseCase available here from com.example.dictionary.domain.

Line 8

```
import kotlinx.coroutines.CancellationException
```

Makes CancellationException available here from kotlinx.coroutines.

Line 9

```
import kotlinx.coroutines.CoroutineScope
```

Makes CoroutineScope available here from kotlinx.coroutines.

Line 10

```
import kotlinx.coroutines.Job
```

Makes Job available here from kotlinx.coroutines.

Line 11

```
import kotlinx.coroutines.flow.MutableSharedFlow
```

Makes MutableSharedFlow available here from kotlinx.coroutines.flow.

Line 12

```
import kotlinx.coroutines.flow.MutableStateFlow
```

Makes MutableStateFlow available here from kotlinx.coroutines.flow.

Line 13

```
import kotlinx.coroutines.flow.SharedFlow
```

Makes SharedFlow available here from kotlinx.coroutines.flow.

Line 14

```
import kotlinx.coroutines.flow.StateFlow
```

Makes StateFlow available here from kotlinx.coroutines.flow.

Line 15

```
import kotlinx.coroutines.launch
```

Makes launch available here from kotlinx.coroutines.

Line 16

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 17

```
// Module 7: this state holder uses plain Kotlin and Flow
 on every platform.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 18

```
class DictionaryViewModel(
```

Defines the shared screen-state holder. Its dependencies are passed into the constructor below.

Line 19

```
private val searchWordUseCase: SearchWordUseCase,
```

Receives/keeps searchWordUseCase as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 20

```
private val historyUseCase: HistoryUseCase,
```

Receives/keeps historyUseCase as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 21

```
private val slangPreferenceUseCase:
SlangPreferenceUseCase,
```

Receives/keeps slangPreferenceUseCase as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 22

```
private val scope: CoroutineScope
```

Receives/keeps scope as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 23

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 24

```
private var searchJob: Job? = null
```

Keeps the current search coroutine so a newer search can cancel it.

Line 25

```
private val _uiState =
MutableStateFlow<UiState>(UiState.Idle)
```

Owns the writable result state. Idle is its first value.

Line 26

```
val uiState: StateFlow<UiState> = _uiState
```

Exposes the current result state for observation; callers are not given the MutableStateFlow API.

Line 27

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 28

```
private val _searchHistory =
MutableStateFlow<List<HistoryItem>>(emptyList())
```

Starts the observable history as an empty list until Room sends saved rows.

Line 29

```
val searchHistory: StateFlow<List<HistoryItem>> =
_searchHistory
```

Exposes the history stream to the UI as read-only StateFlow.

Line 30

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 31

```
private val _showSlang = MutableStateFlow(false)
```

Starts the local preference state as false until its saved value arrives.

Line 32

```
val showSlang: StateFlow<Boolean> = _showSlang
```

Exposes the current slang preference to the UI.

Line 33

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 34

```
// Module 7: messages happen once; screen values belong
in StateFlow.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 35

```
private val _events = MutableSharedFlow<String>()
```

Creates the writable event stream with no replay buffer. Used for one-time messages.

Line 36

```
val events: SharedFlow<String> = _events
```

Exposes those events for collection by App's snackbar handler.

Line 37

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 38

```
init {
```

Starts the following observers when this ViewModel object is created.

Line 39

```
scope.launch {
```

Starts a coroutine inside the provided scope. It can wait for network/storage work without blocking the calling UI code.

Line 40

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 41

```
historyUseCase.observeHistory().collect {
_searchHistory.value = it }
```

Keeps listening to history changes and assigns each new list to the observable UI value.

Line 42

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 43

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 44

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 45

```
_events.emit("Could not read saved history.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 46

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 47

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 48

```
scope.launch {
```

Starts a coroutine inside the provided scope. It can wait for network/storage work without blocking the calling UI code.

Line 49

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 50

```
slangPreferenceUseCase.observeShowSlang().collect {
_showSlang.value = it }
```

Keeps listening to DataStore’s Boolean setting and updates the exposed state.

Line 51

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 52

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 53

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 54

```
_events.emit("Could not read your saved preference.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 55

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 56

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 57

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 58

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 59

```
fun searchWord(word: String, forceRefresh: Boolean =
false) {
```

Starts a word lookup; forceRefresh defaults to false so saved data can be used.

Line 60

```
if (word.isBlank()) return
```

Ignores empty/space-only input before starting any search work.

Line 61

```
searchJob?.cancel()
```

Cancels the previous search if one exists, so its late answer cannot replace the new search.

Line 62

```
searchJob = scope.launch {
```

Starts the new asynchronous search and retains its Job handle.

Line 63

```
val previousState = _uiState.value
```

Remembers the displayed state in case an explicit refresh fails.

Line 64

```
_uiState.value = UiState.Loading
```

Signals the screen to show its loading indicator.

Line 65

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 66

```
val result = searchWordUseCase(word, forceRefresh)
```

Calls the domain operation and waits without blocking for its result.

Line 67

```
if (result == null) {
```

The repository could not obtain a usable response; select the error/fallback behavior.

Line 68

```
if (forceRefresh && previousState is UiState.Success) {
```

When refreshing an already successful result, keep that result on failure.

Line 69

```
_uiState.value = previousState
```

Restores the earlier successful result instead of replacing it with an error.

Line 70

```
_events.emit("Could not refresh. Showing the saved
definition.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 71

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 72

```
_uiState.value = UiState.Error("Could not search. Check
your connection and try again.")
```

Publishes this readable error message as the current result-screen state. Compose observes the new value and draws its Error branch.

Line 73

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 74

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 75

```
showResult(word, result)
```

Turns this result into Success or a no-definitions Error.

Line 76

```
// Module 9: display cached data first, then refresh in
the background.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 77

```
if (result.isCached) {
```

A saved result was just displayed; now try to update it from the network.

Line 78

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 79

```
val fresh = searchWordUseCase(word, forceRefresh = true)
```

Bypasses the saved-word lookup for this background refresh request.

Line 80

```
if (fresh != null && hasDefinitions(fresh)) {
```

Only replace the displayed cached data when fresh definitions actually exist.

Line 81

```
showResult(word, fresh)
```

Publishes fresh definitions after the saved definitions were already usable.

Line 82

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 83

```
_events.emit("Showing saved definitions. A refresh is
unavailable.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 84

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 85

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 86

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 87

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 88

```
_events.emit("Showing saved definitions. A refresh is
unavailable.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 89

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 90

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 91

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 92

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 93

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 94

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 95

```
_uiState.value = UiState.Error("Could not load or save
this word. Please try again.")
```

Publishes this readable error message as the current result-screen state. Compose observes the new value and draws its Error branch.

Line 96

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 97

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 98

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 99

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 100

```
private fun hasDefinitions(result: DictionaryResult):
Boolean {
```

Small helper that asks whether either source supplied definitions.

Line 101

```
return result.formalEntries.isNotEmpty() ||
result.urbanEntries.isNotEmpty()
```

True when at least one dictionary list contains entries.

Line 102

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 103

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 104

```
private fun showResult(word: String, result:
DictionaryResult) {
```

Converts the domain answer into the state the result screen understands.

Line 105

```
_uiState.value = if (!hasDefinitions(result) &&
result.didYouMeanSuggestion == null) {
```

Chooses Error for no entries/no suggestion, otherwise Success.

Line 106

```
UiState.Error("No definitions found for
'${word.trim()}'.")
```

Creates the readable no-definitions state for this word.

Line 107

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 108

```
UiState.Success(result.formalEntries,
result.urbanEntries, result.didYouMeanSuggestion)
```

Publishes the two definition lists and optional spelling suggestion together.

Line 109

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 110

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 111

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 112

```
fun toggleFavorite(word: String) {
```

Receives a star-button action from App.

Line 113

```
scope.launch {
```

Starts a coroutine inside the provided scope. It can wait for network/storage work without blocking the calling UI code.

Line 114

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 115

```
historyUseCase.toggleFavorite(word)
```

Requests a persistent favorite update. History will redraw when the database Flow emits.

Line 116

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 117

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 118

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 119

```
_events.emit("Could not save this favorite.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 120

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 121

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 123

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 124

```
fun saveShowSlang(enabled: Boolean) {
```

Receives the new checkbox value from App.

Line 125

```
scope.launch {
```

Starts a coroutine inside the provided scope. It can wait for network/storage work without blocking the calling UI code.

Line 126

```
try {
```

Begins work that can fail; the following catch/finally blocks define its recovery/cleanup.

Line 127

```
slangPreferenceUseCase.saveShowSlang(enabled)
```

Saves the Boolean through the preferences use case.

Line 128

```
_events.emit("Preference saved.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 129

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 130

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 131

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 132

```
_events.emit("Could not save your preference.")
```

Sends this one-time message to collectors. It is not stored as the lasting result state.

Line 133

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 134

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 135

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 136

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

6. DictionaryRepository.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/DictionaryRepository.kt

Checks Room before network requests, converts responses, saves useful definitions and exposes history.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import com.example.dictionary.domain.DictionaryEntry
```

Makes DictionaryEntry available here from com.example.dictionary.domain.

Line 4

```
import com.example.dictionary.domain.DictionaryResult
```

Makes DictionaryResult available here from com.example.dictionary.domain.

Line 5

```
import com.example.dictionary.domain.DictionaryRepository
```

Makes DictionaryRepository available here from com.example.dictionary.domain.

Line 6

```
import com.example.dictionary.domain.HistoryItem
```

Makes HistoryItem available here from com.example.dictionary.domain.

Line 7

```
import com.example.dictionary.data.local.HistoryDao
```

Makes HistoryDao available here from com.example.dictionary.data.local.

Line 8

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 9

```
import kotlinx.coroutines.flow.map
```

Makes map available here from kotlinx.coroutines.flow.

Line 10

```
import kotlinx.serialization.encodeToString
```

Makes encodeToString available here from kotlinx.serialization.

Line 11

```
import kotlinx.coroutines.CancellationException
```

Makes CancellationException available here from kotlinx.coroutines.

Line 12

```
import io.ktor.client.*
```

Makes * available from io.ktor.client. A star imports accessible declarations from that package.

Line 13

```
import io.ktor.client.call.*
```

Makes * available from io.ktor.client.call. A star imports accessible declarations from that package.

Line 14

```
import io.ktor.client.request.*
```

Makes * available from io.ktor.client.request. A star imports accessible declarations from that package.

Line 15

```
import io.ktor.http.*
```

Makes * available from io.ktor.http. A star imports accessible declarations from that package.

Line 16

```
import kotlinx.serialization.Serializable
```

Makes Serializable available here from kotlinx.serialization.

Line 17

```
import kotlinx.serialization.json.*
```

Makes * available from kotlinx.serialization.json. A star imports accessible declarations from that package.

Line 18

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 19

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 20

```
private data class UrbanResponse(val list:
List<UrbanDefinition>)
```

Models the top-level response containing a list of slang definitions.

Line 21

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 22

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 23

```
private data class UrbanDefinition(
```

Models a slang entry’s fields so JSON can be decoded into Kotlin values.

Line 24

```
val definition: String,
```

The meaning to display.

Line 25

```
val word: String,
```

The word associated with this object.

Line 26

```
val example: String = ""
```

An optional example sentence; empty when unavailable.

Line 27

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 28

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 29

```
// Module 6: App supplies the client and API key through
the constructor.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 30

```
class DictionaryRepositoryImpl(
```

Implements the domain repository promise using injected Room operations and Ktor.

Line 31

```
private val dao: HistoryDao,
```

Receives/keeps dao as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 32

```
private val client: HttpClient,
```

Receives/keeps client as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 33

```
private val apiKey: String
```

Receives/keeps apiKey as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 34

```
) : DictionaryRepository {
```

Finishes the injected constructor and declares that this class implements the domain DictionaryRepository interface.

Line 35

```
private val json = Json { ignoreUnknownKeys = true }
```

Creates a reusable JSON converter that ignores fields the app does not model.

Line 36

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 37

```
override suspend fun fetchDefinitions(word: String,
forceRefresh: Boolean): DictionaryResult? {
```

Implements a possibly-pausing dictionary lookup. A null answer represents an unusable failed lookup.

Line 38

```
val sanitizedInput = word.trim().lowercase()
```

Removes surrounding spaces and converts letters to lowercase for consistent lookups/keys.

Line 39

```
// Module 9: read the local database first, unless
explicitly refreshing.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 40

```
if (!forceRefresh) {
```

Use saved data only when the caller has not explicitly requested a refresh.

Line 41

```
val cached = dao.findWord(sanitizedInput)
```

Reads the saved row for this normalized word.

Line 42

```
if (cached != null) {
```

There is an existing row, so return its definitions before making online requests.

Line 43

```
return json.decodeFromString<DictionaryResult>(cached.res
ultJson).copy(isCached = true)
```

Converts saved JSON back to DictionaryResult and marks this returned copy as cached.

Line 44

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 45

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 46

```
var failedSources = 0
```

Counts dictionary sources that failed during this network lookup.

Line 47

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 48

```
val urbanEntries = try {
```

Attempts Urban Dictionary independently so one service’s failure need not discard the other service’s answer.

Line 49

```
val httpResponse =
client.get("https://api.urbandictionary.com/v0/define") {
```

Starts the Urban Dictionary GET request; the following parameter contains the word.

Line 50

```
parameter("term", sanitizedInput)
```

Adds the word as the term query parameter. Ktor handles query-parameter encoding.

Line 51

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 52

```
if (httpResponse.status != HttpStatusCode.OK) {
```

Checks the HTTP response status rather than assuming every response contains valid definitions.

Line 53

```
error("Urban Dictionary is unavailable.")
```

Raises a failure that the enclosing catch handles as a failed dictionary source.

Line 54

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 55

```
val response: UrbanResponse = httpResponse.body()
```

Uses Ktor’s JSON converter to read the response into the typed slang response model.

Line 56

```
response.list.take(3).map {
```

Keeps up to three slang entries and converts each API model into the app’s DictionaryEntry model.

Line 57

```
DictionaryEntry(
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 58

```
word = it.word,
```

Copies the word supplied by the slang entry.

Line 59

```
source = "Urban Dictionary",
```

Labels the entry with the dictionary that supplied it.

Line 60

```
definition = it.definition.replace("[", "").replace("]",
"),
```

Removes Urban Dictionary’s square-bracket link markers from the displayed definition.

Line 61

```
example = it.example.replace("[", "").replace("]", "")
```

Removes those markers from the example sentence too.

Line 62

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 63

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 64

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 65

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 66

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 67

```
failedSources++
```

Adds one to the number of failed sources.

Line 68

```
println("Dictionary request failed:
${e::class.simpleName}")
```

Prints the error's class name for diagnosis without printing a request URL/API key.

Line 69

```
emptyList()
```

Uses an empty list as this block's answer; callers can safely handle no entries.

Line 70

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 71

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 72

```
var mwDidYouMean: String? = null
```

Starts with no spelling correction; the formal API may supply one.

Line 73

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 74

```
val formalEntries = try {
```

Attempts the formal dictionary request after the slang request.

Line 75

```
val httpResponse = client.get(
```

Stores a value/reference named httpResponse from the expression on the right.

Line 76

```
"https://dictionaryapi.com/api/v3/references/collegiate/j
son/${sanitizedInput.encodeURLPathPart()}"
```

Builds the formal dictionary URL and encodes the word for its URL path.

Line 77

```
) {
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 78

```
parameter("key", apiKey)
```

Adds the repository's injected API key to the formal dictionary request.

Line 79

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 80

```
if (httpResponse.status != HttpStatusCode.OK) {
```

Checks the HTTP response status rather than assuming every response contains valid definitions.

Line 81

```
error("Merriam-Webster is unavailable.")
```

Raises a failure that the enclosing catch handles as a failed dictionary source.

Line 82

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 83

```
// This API returns either spelling strings OR definition
 objects.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 84

```
// Only inspect the first item; serialization handles the
 definition fields.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 85

```
val response: JsonArray = httpResponse.body()
```

Reads the formal response as an array because this API returns either objects or spelling strings.

Line 86

```
if (response.firstOrNull() is JsonPrimitive) {
```

Distinguishes a suggestion-string response from a definition-object response.

Line 87

```
mwDidYouMean = response.first().jsonPrimitive.content
```

Keeps the first suggested word for the clickable correction banner.

Line 88

```
emptyList()
```

Uses an empty list as this block’s answer; callers can safely handle no entries.

Line 89

```
} else {
```

Runs the alternative branch when the earlier condition did not match.

Line 90

```
val definitions = json.decodeFromJsonElement<List<FormalD
efinition>>(response)
```

Uses typed serialization for the formal objects after checking which array format was returned.

Line 91

```
val entries = mutableListOf<DictionaryEntry>()
```

Creates a temporary list to collect the formal definitions converted into app models.

Line 92

```
for (item in definitions.take(3)) {
```

Processes up to three formal dictionary items.

Line 93

```
if (item.shortdef.isNotEmpty()) {
```

Skips formal items that do not have a short definition.

Line 94

```
entries.add(DictionaryEntry(
```

Adds one converted formal definition to the temporary result list.

Line 95

```
word = sanitizedInput,
```

Uses the normalized search term for this formal entry.

Line 96

```
source = "Merriam-Webster",
```

Labels the entry with the dictionary that supplied it.

Line 97

```
partOfSpeech = item.fl,
```

Copies the API’s fl field, such as noun, into the app’s partOfSpeech property.

Line 98

```
definition = item.shortdef.first()
```

Uses the first short definition from this formal item.

Line 99

```
))
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 100

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 101

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 102

```
entries
```

The last expression of this try branch is its result: return the accumulated formal-entry list as formalEntries.

Line 103

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 104

```
} catch (e: CancellationException) {
```

Handles coroutine cancellation separately so stopping a search is not treated as an ordinary failed API call.

Line 105

```
throw e
```

Passes cancellation back up so the coroutine really stops. Swallowing it could let stale work continue.

Line 106

```
} catch (e: Exception) {
```

Handles a failure from the matching try block. The following lines choose a safe result/message rather than leaving it unhandled.

Line 107

```
failedSources++
```

Adds one to the number of failed sources.

Line 108

```
println("Dictionary request failed:
${e::class.simpleName}")
```

Prints the error's class name for diagnosis without printing a request URL/API key.

Line 109

```
emptyList()
```

Uses an empty list as this block's answer; callers can safely handle no entries.

Line 110

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 111

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 112

```
if (formalEntries.isEmpty() && urbanEntries.isEmpty() &&
failedSources > 0 && mwDidYouMean == null) {
```

Keeps the first suggested word for the clickable correction banner.

Line 113

```
return null
```

Sends no usable result to the caller. The ViewModel decides the error screen.

Line 114

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 115

```
val result = DictionaryResult(formalEntries,
urbanEntries, mwDidYouMean)
```

Packages formal entries, slang entries and the optional correction into one named answer.

Line 116

```
// Save useful results, including when only one
dictionary is reachable.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 117

```
if (formalEntries.isNotEmpty() ||
urbanEntries.isNotEmpty()) {
```

Save only actual definitions; a spelling suggestion alone is not a saved dictionary result.

Line 118

```
val snippet = formalEntries.firstOrNull()?.definition
```

Prefers a formal definition for the history preview; the next line falls back to slang.

Line 119

```
?: urbanEntries.first().definition
```

Uses the first slang definition if there is no formal one. The earlier condition ensures one list has data.

Line 120

```
dao.saveWord(sanitizedInput, snippet,
json.encodeToString(result))
```

Serializes the result lists into JSON and stores the word/preview/result through Room.

Line 121

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 122

```
return result
```

Returns the lookup answer to the use case, then the ViewModel.

Line 123

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 124

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 125

```
// Modules 7 and 9: Room emits history again whenever
saved data changes.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 126

```
override fun observeHistory(): Flow<List<HistoryItem>> {
```

Exposes saved history as a stream, not just one list read once.

Line 127

```
return dao.observeHistory().map { savedWords ->
```

Transforms each new list emitted by the Room query into UI-friendly history items.

Line 128

```
savedWords.map { saved ->
```

Converts every saved row in the latest emitted list.

Line 129

```
val result =
json.decodeFromString<DictionaryResult>(saved.resultJson)
```

Reconstructs the definition lists from the row's saved JSON text.

Line 130

```
HistoryItem(saved.word, saved.snippet,
result.formalEntries,
```

Starts the history item with the word, preview and formal definitions.

Line 131

```
result.urbanEntries, saved.isFavorite)
```

Includes saved slang definitions and the current database favorite flag.

Line 132

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 133

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 134

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 135

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 136

```
override suspend fun toggleFavorite(word: String) {
```

Implements the persistent star/unstar operation promised by the interface.

Line 137

```
dao.toggleFavorite(word.trim().lowercase())
```

Normalizes the word and forwards the update to Room.

Line 138

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 139

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 140

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 141

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 142

```
@Serializable
```

Asks the serialization compiler plugin to generate JSON conversion support for the next data class.

Line 143

```
private data class FormalDefinition(
```

Models only the formal API fields used by this app; other fields are ignored.

Line 144

```
val fl: String = "",
```

The API field fl contains a part of speech; missing values become an empty String.

Line 145

```
val shortdef: List<String> = emptyList()
```

The formal API’s list of short definitions; missing values become an empty list.

Line 146

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

7. HistoryUseCase.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/HistoryUseCase.kt

A domain-facing way to observe history and change a favorite through the repository.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
class HistoryUseCase(private val repository:
DictionaryRepository) {
```

Defines HistoryUseCase as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 6

```
fun observeHistory(): Flow<List<HistoryItem>> =
repository.observeHistory()
```

Defines observeHistory(). Inputs are inside parentheses; the return type describes its answer.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
suspend fun toggleFavorite(word: String) {
```

Defines toggleFavorite(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 9

```
repository.toggleFavorite(word)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

8. PreferencesRepository.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/PreferencesRepository.kt

The contract for observing/saving the slang-default Boolean.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
interface PreferencesRepository {
```

Defines PreferencesRepository as an interface of promised operations.

Line 6

```
fun observeShowSlang(): Flow<Boolean>
```

Defines observeShowSlang(). Inputs are inside parentheses; the return type describes its answer.

Line 7

```
suspend fun saveShowSlang(enabled: Boolean)
```

Defines saveShowSlang(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 8

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

9. SlangPreferenceUseCase.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/domain/SlangPreferenceUseCase.kt

Forwards preference operations through the preference interface.

Line 1

```
package com.example.dictionary.domain
```

Places this file in the com.example.dictionary.domain namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 4

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 5

```
class SlangPreferenceUseCase(private val preferences:
PreferencesRepository) {
```

Defines SlangPreferenceUseCase as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 6

```
fun observeShowSlang(): Flow<Boolean> =
preferences.observeShowSlang()
```

Defines observeShowSlang(). Inputs are inside parentheses; the return type describes its answer.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
suspend fun saveShowSlang(enabled: Boolean) {
```

Defines saveShowSlang(). It can pause because it is suspend. Inputs are inside parentheses; the return type describes its answer.

Line 9

```
preferences.saveShowSlang(enabled)
```

Calls this named operation using the arguments shown here/below. Read the operation’s definition or the file summary for its responsibility.

Line 10

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 11

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

10. HistoryEntity.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/local/HistoryEntity.kt

Defines one saved-search database row, including its primary-key word.

Line 1

```
package com.example.dictionary.data.local
```

Places this file in the com.example.dictionary.data.local namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.room3.Entity
```

Makes Entity available here from androidx.room3.

Line 4

```
import androidx.room3.PrimaryKey
```

Makes PrimaryKey available here from androidx.room3.

Line 5

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 6

```
// Module 9: one saved search per word. Room stores this
in SQLite.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 7

```
@Entity(tableName = "search_history")
```

Registers the next Kotlin data class as a Room database table.

Line 8

```
data class HistoryEntity(
```

Defines HistoryEntity as a data container.

Line 9

```
@PrimaryKey val word: String,
```

The word associated with this object.

Line 10

```
val snippet: String,
```

A short preview of a saved definition for History.

Line 11

```
val resultJson: String,
```

Serialized definition lists stored as one database text value.

Line 12

```
val isFavorite: Boolean = false,
```

Whether this history word is starred; false is the default.

Line 13

```
val searchOrder: Long = 0
```

A numeric order used to display newer saved/refreshed searches first.

Line 14

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

11. HistoryDao.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/local/HistoryDao.kt

Room’s saved-word operations. SQL chooses rows, updates definitions and flips stars.

Line 1

```
package com.example.dictionary.data.local
```

Places this file in the com.example.dictionary.data.local namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.room3.Dao
```

Makes Dao available here from androidx.room3.

Line 4

```
import androidx.room3.Query
```

Makes Query available here from androidx.room3.

Line 5

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 6

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 7

```
@Dao
```

Asks Room to generate the database implementation for the following DAO interface.

Line 8

```
interface HistoryDao {
```

Defines HistoryDao as an interface of promised operations.

Line 9

```
@Query("SELECT * FROM search_history ORDER BY isFavorite
DESC, searchOrder DESC")
```

Defines the history query: favorites first, then most recently saved/refreshed rows.

Line 10

```
fun observeHistory(): Flow<List<HistoryEntity>>
```

Room returns a Flow that emits again when the table changes.

Line 11

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 12

```
@Query("SELECT * FROM search_history WHERE word = :word")
```

Defines a lookup for exactly one saved word.

Line 13

```
suspend fun findWord(word: String): HistoryEntity?
```

Returns that row, or null when the word is not stored.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
// Updating a definition keeps the user's existing
favorite flag.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 16

```
@Query("""
```

Attaches this SQL statement to the DAO function immediately below it. Room checks it and generates the database call.

Line 17

```
INSERT INTO search_history (word, snippet, resultJson,
isFavorite, searchOrder)
```

Names the columns written for a saved search.

Line 18

```
VALUES (:word, :snippet, :resultJson, 0,
```

Supplies query parameters, sets a new row's favorite to false, and begins calculating its order.

Line 19

```
(SELECT COALESCE(MAX(searchOrder), 0) + 1 FROM
search_history))
```

Uses the highest existing order plus one; COALESCE uses zero for an empty table.

Line 20

```
ON CONFLICT(word) DO UPDATE SET
```

If this primary-key word exists, update that row instead of creating a duplicate.

Line 21

```
snippet = excluded.snippet,
```

Replace the existing row's preview with the new preview.

Line 22

```
resultJson = excluded.resultJson,
```

Replace saved definitions with the new serialized result.

Line 23

```
searchOrder = excluded.searchOrder
```

Move the saved/refreshed word to the newest order within its favorite group. isFavorite is deliberately not overwritten.

Line 24

```
""")
```

A text/String value used by the surrounding call, query or configuration. Its role is determined by that surrounding operation.

Line 25

```
suspend fun saveWord(word: String, snippet: String,
resultJson: String)
```

Declares the parameters Room binds into the insertion/update query above.

Line 26

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 27

```
@Query("UPDATE search_history SET isFavorite = NOT
isFavorite WHERE word = :word")
```

Flips the favorite flag only for the matching word.

Line 28

```
suspend fun toggleFavorite(word: String)
```

Runs the favorite update asynchronously through Room.

Line 29

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

12. DictionaryDatabase.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/local/DictionaryDatabase.kt

Registers the entity and exposes its DAO. Room generates the constructor implementation.

Line 1

```
package com.example.dictionary.data.local
```

Places this file in the com.example.dictionary.data.local namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.room3.ConstructedBy
```

Makes ConstructedBy available here from androidx.room3.

Line 4

```
import androidx.room3.Database
```

Makes Database available here from androidx.room3.

Line 5

```
import androidx.room3.RoomDatabase
```

Makes RoomDatabase available here from androidx.room3.

Line 6

```
import androidx.room3.RoomDatabaseConstructor
```

Makes RoomDatabaseConstructor available here from androidx.room3.

Line 7

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 8

```
@Database(entities = [HistoryEntity::class], version = 1)
```

Registers HistoryEntity in database schema version 1.

Line 9

```
@ConstructedBy(DictionaryDatabaseConstructor::class)
```

Connects this database to the constructor Room generates for the platforms.

Line 10

```
abstract class DictionaryDatabase : RoomDatabase() {
```

Defines DictionaryDatabase as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 11

```
abstract fun historyDao(): HistoryDao
```

Defines historyDao(). Inputs are inside parentheses; the return type describes its answer.

Line 12

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 13

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 14

```
// Room generates the actual implementation for each
target.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 15

```
@Suppress("KotlinNoActualForExpect")
```

Acknowledges a specific compiler warning/opt-in for the API used here. It does not perform a search or alter stored data.

Line 16

```
expect object DictionaryDatabaseConstructor :
RoomDatabaseConstructor<DictionaryDatabase> {
```

Declares the database constructor that Room will generate for each target. This is generated-library setup, not a manually implemented dictionary feature.

Line 17

```
override fun initialize(): DictionaryDatabase
```

Defines initialize(). This fulfills the matching interface/parent operation. Inputs are inside parentheses; the return type describes its answer.

Line 18

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

13. DictionaryPreferences.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/DictionaryPreferences.kt

Reads and saves one small setting using DataStore.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStore
```

Makes DataStore available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.preferences.core.Preferences
```

Makes Preferences available here from androidx.datastore.preferences.core.

Line 5

```
import
androidx.datastore.preferences.core.booleanPreferencesKey
```

Makes booleanPreferencesKey available here from androidx.datastore.preferences.core.

Line 6

```
import androidx.datastore.preferences.core.edit
```

Makes edit available here from androidx.datastore.preferences.core.

Line 7

```
import
com.example.dictionary.domain.PreferencesRepository
```

Makes PreferencesRepository available here from com.example.dictionary.domain.

Line 8

```
import kotlinx.coroutines.flow.Flow
```

Makes Flow available here from kotlinx.coroutines.flow.

Line 9

```
import kotlinx.coroutines.flow.map
```

Makes map available here from kotlinx.coroutines.flow.

Line 10

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 11

```
// Module 9: DataStore is for a small setting;
definitions belong in Room.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 12

```
class DictionaryPreferences(private val dataStore:
DataStore<Preferences>) : PreferencesRepository {
```

Implements the preference interface using the injected DataStore object.

Line 13

```
private val showSlangKey =
booleanPreferencesKey("show_slang")
```

Defines the Boolean key named show_slang used in the preference file.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
override fun observeShowSlang(): Flow<Boolean> {
```

Returns a stream of the saved Boolean preference.

Line 16

```
return dataStore.data.map { preferences ->
preferences[showSlangKey] ?: false }
```

Reads the key from each preference snapshot, using false when the user has never saved it.

Line 17

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 18

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 19

```
override suspend fun saveShowSlang(enabled: Boolean) {
```

Declares the operation to save a new Boolean preference.

Line 20

```
dataStore.edit { preferences -> preferences[showSlangKey]
 = enabled }
```

Changes only this preference key in a DataStore update.

Line 21

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 22

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

14. AppStorage.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/AppStorage.kt

A small container holding the database and preferences supplied by a platform entry point.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import androidx.datastore.core.DataStore
```

Makes DataStore available here from androidx.datastore.core.

Line 4

```
import androidx.datastore.preferences.core.Preferences
```

Makes Preferences available here from androidx.datastore.preferences.core.

Line 5

```
import
com.example.dictionary.data.local.DictionaryDatabase
```

Makes DictionaryDatabase available here from com.example.dictionary.data.local.

Line 6

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 7

```
// Platform entry points create these once and inject
them into App.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 8

```
class AppStorage(
```

Defines AppStorage as a type used by the surrounding layer. Constructor inputs, if any, follow here.

Line 9

```
val database: DictionaryDatabase,
```

Declares the database value and its type for this object or operation. A question mark allows null; a default is used if no value is supplied.

Line 10

```
val preferences: DataStore<Preferences>
```

Receives/keeps preferences as a dependency used by this class. Passing it from outside is constructor injection; this line does not itself execute its operations.

Line 11

```
)
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

15. HttpClientFactory.kt (commonMain)

shared/src/commonMain/kotlin/com/example/dictionary/data/HttpClientFactory.kt

Declares the platform factory and centralizes JSON, logs and timeouts.

Line 1

```
package com.example.dictionary.data
```

Places this file in the com.example.dictionary.data namespace so related declarations can be found without name conflicts.

Line 2

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 3

```
import io.ktor.client.HttpClient
```

Makes HttpClient available here from io.ktor.client.

Line 4

```
import io.ktor.client.HttpClientConfig
```

Makes HttpClientConfig available here from io.ktor.client.

Line 5

```
import io.ktor.client.plugins.HttpTimeout
```

Makes HttpTimeout available here from io.ktor.client.plugins.

Line 6

```
import io.ktor.client.plugins.contentnegotiation.ContentN
egotiation
```

Makes ContentNegotiation available here from io.ktor.client.plugins.contentnegotiation.

Line 7

```
import io.ktor.client.plugins.logging.Logging
```

Makes Logging available here from io.ktor.client.plugins.logging.

Line 8

```
import io.ktor.client.plugins.logging.LogLevel
```

Makes LogLevel available here from io.ktor.client.plugins.logging.

Line 9

```
import io.ktor.serialization.kotlinx.json.json
```

Makes json available here from io.ktor.serialization.kotlinx.json.

Line 10

```
import kotlinx.serialization.json.Json
```

Makes Json available here from kotlinx.serialization.json.

Line 11

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 12

```
// Module 6: each platform chooses its own networking
engine.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 13

```
expect fun createHttpClient(): HttpClient
```

Shared declaration: each platform must supply an actual HTTP client factory.

Line 14

```
[blank line]
```

Blank line: separates related code visually; it performs no operation.

Line 15

```
// Module 9: the plugins are configured once in shared
code.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 16

```
fun HttpClientConfig<*>.configureDictionaryClient() {
```

An extension function: adds shared configuration steps to any Ktor client engine configuration.

Line 17

```
install(ContentNegotiation) {
```

Adds the plugin that converts HTTP response bodies using the configured serializer.

Line 18

```
json(Json { ignoreUnknownKeys = true })
```

Uses Kotlin JSON serialization and ignores extra response fields.

Line 19

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 20

```
install(Logging) {
```

Adds Ktor’s request/response logging support.

Line 21

```
level = LogLevel.INFO
```

Logs general request/response information rather than response bodies.

Line 22

```
// The other API's URL contains a key, so log only
key-free requests.
```

Comment: explains or attributes the nearby code; it is not an app instruction executed at runtime.

Line 23

```
filter { request -> request.url.host ==
"api.urbandictionary.com" }
```

Logs only Urban Dictionary traffic; formal request URLs contain the API key.

Line 24

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 25

```
install(HttpTimeout) {
```

Adds time limits so requests do not wait indefinitely.

Line 26

```
requestTimeoutMillis = 10000
```

Limits the whole request to 10,000 milliseconds (10 seconds).

Line 27

```
connectTimeoutMillis = 10000
```

Limits establishing the connection to 10 seconds.

Line 28

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.

Line 29

```
}
```

Closes the surrounding block/call/list opened above. Read the indentation to identify which one; this line has no independent business operation.